Predicción del abandono estudiantil mediante árboles de decisión

Trabajo Práctico 05 — b) Instituto Tecnológico Beltrán (ITB)

Procesamiento de Aprendizaje Automático — Equipo 8
Informe de resultados — 16 de octubre de 2026

Resumen. Se entrenó un árbol de decisión sobre 100 alumnos del ITB para predecir el abandono de la carrera. El modelo final alcanzó una **accuracy de 0,433** en el conjunto de prueba, **por debajo del 0,533** que obtiene el clasificador trivial que siempre predice la clase mayoritaria, y un **ROC-AUC de 0,388** (peor que el azar). Un test de permutación con 200 repeticiones arrojó un **p-valor de 0,164**, por lo que no se puede rechazar la hipótesis nula: **los datos provistos no contienen una relación aprendible** entre los atributos del alumno y su estado final. En consecuencia, no se informa ninguna variable como «la más importante» ni se recomienda usar el modelo para decidir intervenciones. Un experimento de control con datos simulados verifica que el procedimiento está bien implementado y que la limitación es del dataset, no del código.

1. Objetivo y datos

El ITB quiere identificar tempranamente a los estudiantes con riesgo de abandonar la carrera para destinarles tutorías y acompañamiento. La consigna pide: (1) analizar y preparar los datos, (2) entrenar un árbol de decisión, (3) evaluarlo sobre un conjunto de prueba, (4) interpretar qué variables resultan más importantes y (5) elaborar recomendaciones para la institución.

Se trabajó con dos archivos: **TablaPrediccionAbandono-Entrenamiento.xlsx** (100 alumnos con la etiqueta *EstadoFinal*) y **TablaPrediccionAbandono-DatosFinal.xlsx** (100 alumnos sin etiqueta, a predecir). Las 10 variables de entrada son: edad, género, carrera, promedio del primer cuatrimestre, materias aprobadas y desaprobadas, asistencia promedio (%), situación laboral, distancia al instituto en km y participación en tutorías. Todo el análisis se hizo en Python 3.14 con scikit-learn 1.8 y semilla fija (*random_state* = 42) para que los resultados sean reproducibles.

2. Análisis exploratorio (Tarea 1)

El dataset está limpio: **sin valores nulos y sin filas duplicadas**, por lo que no hizo falta imputar ni descartar registros. Las clases están casi balanceadas: **54 abandonos y 46 continuidades**. Esto es importante por dos motivos: no se necesitan técnicas agresivas de rebalanceo, pero fija la vara a superar en 54%, que es lo que acierta un modelo que siempre responde «abandona».

Calculando la entropía a mano, como en el TP 04: H(S) = –[0,54·log■(0,54) + 0,46·log■(0,46)] = **0,9954 bits**. Un valor prácticamente igual a 1 indica máxima incertidumbre inicial; el árbol debería reducirla.

2.1 ¿Hay señal en los datos?

Antes de entrenar comparamos el promedio de cada variable numérica entre los dos grupos. Si una variable sirve para predecir, sus medias deberían diferir:

Variable	Abandonó	Continúa	Diferencia	Dif. % s/ media
Edad	41,69	37,15	4,53	11,45 %
Promedio	7,24	6,95	0,30	4,16 %
Aprobadas	3,70	4,00	0,30	7,72 %
Desaprobadas	4,22	4,52	0,30	6,87 %
Asistencia (%)	68,42	69,70	1,28	1,85 %

Variable	Abandonó	Continúa	Diferencia	Dif. % s/ media
Distancia (km)	10,57	10,24	0,33	3,21 %

El resultado es contraintuitivo y fue la **primera señal de alarma**: quienes abandonaron tuvieron una asistencia del 68,4% contra 69,7% de quienes continuaron —una diferencia de apenas 1,3 puntos—, e incluso un promedio levemente *más alto* (7,24 vs. 6,95). Esperaríamos exactamente lo contrario. Las variables categóricas confirman el diagnóstico: la tasa de abandono se mantiene entre 51% y 57% en todas las categorías de género, situación laboral y participación en tutorías, es decir, prácticamente la tasa general del 54%. **Ninguna categoría discrimina.**

3. Preparación de los datos (Tarea 1)

Se agregaron dos variables derivadas con sentido pedagógico: **TotalMaterias** (aprobadas + desaprobadas) y **TasaAprobacion** (aprobadas / total). La tasa es más informativa que el conteo crudo, porque aprobar 3 de 4 no es lo mismo que aprobar 3 de 9. Con esto X quedó en 12 atributos.

Las 4 variables categóricas se codificaron con **One-Hot Encoding** —y no asignándoles números correlativos— porque numerar categorías inventa un orden que no existe y haría que el árbol produjera cortes sin sentido del tipo «carrera ≤ 1,5». Tras la codificación quedaron **23 columnas**. El preprocesamiento se encapsuló en un *ColumnTransformer* dentro de un *Pipeline*, de modo que se aprenda solo con los datos de entrenamiento y se aplique idénticamente a prueba y a los datos finales: así se evita la fuga de información (*data leakage*). Las variables numéricas no se escalaron porque el árbol parte por umbrales y no por distancias.

Finalmente se partió en **70% entrenamiento (70 alumnos, 54,3% abandono) y 30% prueba (30 alumnos, 53,3% abandono)**, con *stratify* para conservar la proporción de clases en ambos conjuntos.

4. Entrenamiento del árbol (Tarea 2)

4.1 Baseline y sobreajuste

Como referencia obligatoria se entrenó un *DummyClassifier* que siempre predice la clase mayoritaria: su accuracy en prueba es **0,533**. Ese es el número a superar.

Luego, para evidenciar el riesgo de sobreajuste, se entrenó un árbol sin ninguna restricción. Alcanzó **100% de accuracy en entrenamiento y solo 40% en prueba**, con profundidad 14 y 20 hojas: memorizó uno por uno a los 70 alumnos de entrenamiento y fracasó con alumnos nuevos. Es el caso de manual de por qué hay que podar.

4.2 Búsqueda de hiperparámetros

En lugar de elegir la profundidad «a ojo», se exploraron **360 combinaciones** con *GridSearchCV* y validación cruzada estratificada de 5 pliegues, optimizando **F1 sobre la clase «Abandonó»** (y no accuracy, porque lo que interesa es detectar bien a los alumnos en riesgo). La mejor configuración fue:

Hiperparámetro	Valor elegido	Hiperparámetro	Valor elegido
criterion	gini	min_samples_split	5
max_depth	5	class_weight	None
min_samples_leaf	1	Mejor F1 en CV	0,587

El árbol resultante tiene **profundidad 5 y 8 hojas**. Sin embargo, su accuracy por pliegue fue [0,357 — 0,571 — 0,786 — 0,571 — 0,429]: media **0,543** con un desvío de **0,147**. Un desvío tan alto significa que el resultado depende casi por completo de qué alumnos cayeron en cada pliegue; **el modelo no es estable**.

5. Evaluación en el conjunto de prueba (Tarea 3)

Métrica	Valor	Lectura
Accuracy	0,433	Por debajo del baseline (0,533)
Precision (Abandono)	0,455	Menos de la mitad de las alertas son correctas
Recall (Abandono)	0,312	Detecta solo 5 de 16 abandonos reales
F1 (Abandono)	0,370	Combinación pobre de ambas
ROC-AUC	0,388	Menor a 0,50: peor que tirar una moneda
Baseline (clase mayoritaria)	0,533	El modelo pierde por 10 puntos

La matriz de confusión muestra el detalle sobre los 30 alumnos de prueba. El error más caro para la institución son los **11 falsos negativos**: alumnos que efectivamente abandonaron y que el modelo clasificó como «continúa», es decir, 11 chicos que se habrían quedado sin acompañamiento. Frente a ellos, solo **5 abandonos detectados** y **6 alertas en falso**.

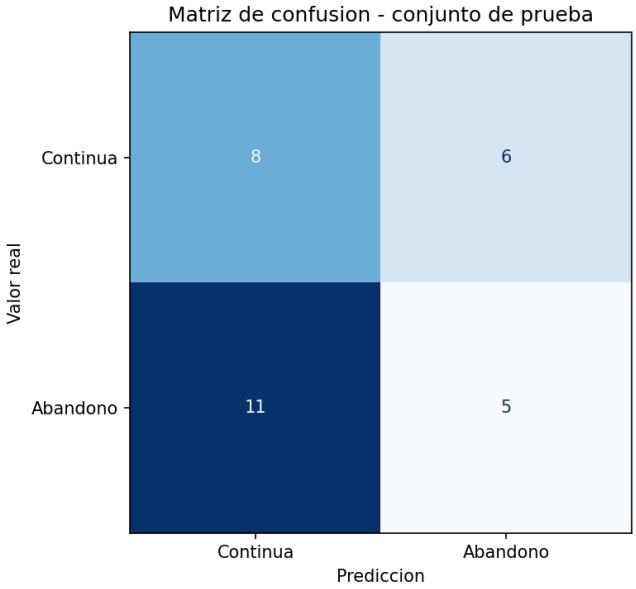


Figura 1. Matriz de confusión sobre el conjunto de prueba (30 alumnos). Filas = valor real, columnas = predicción.

6. ¿El modelo aprendió algo real? Test de permutación

Este es el punto decisivo del trabajo. La idea es simple: si el modelo captura un patrón verdadero, debería funcionar **mucho mejor con las etiquetas reales que con etiquetas mezcladas al azar**. Se barajó la variable objetivo 200 veces y en cada repetición se midió la accuracy por validación cruzada.

Escenario	Accuracy (CV)
Modelo con etiquetas REALES	0,560
Etiquetas AL AZAR — media de 200 repeticiones	0,504
Etiquetas AL AZAR — máximo obtenido	0,640
p-valor del test de permutación	0,164

Conclusión estadística. Con p = 0,164 > 0,05 **no se puede rechazar la hipótesis nula**. El modelo no rinde mejor que el azar. De hecho, el mejor resultado obtenido mezclando las etiquetas al azar (0,640) fue **superior** al obtenido con las etiquetas verdaderas (0,560). Esto indica que los datos provistos no contienen una relación aprendible entre los atributos del alumno y su estado final.

7. Interpretación del árbol (Tarea 4)

La consigna pide identificar qué variables son más importantes. Técnicamente el árbol entrega una respuesta: la raíz es **Asistencia ≤ 96,96** y el ranking de importancia por reducción de impureza queda encabezado por **Promedio (0,351)**, seguido de **Asistencia (0,205)**, la carrera de Ciencia de Datos (0,163), DistanciaKm (0,157) y Edad (0,125). De las 23 columnas disponibles, el árbol **no usó 18** en ningún corte.

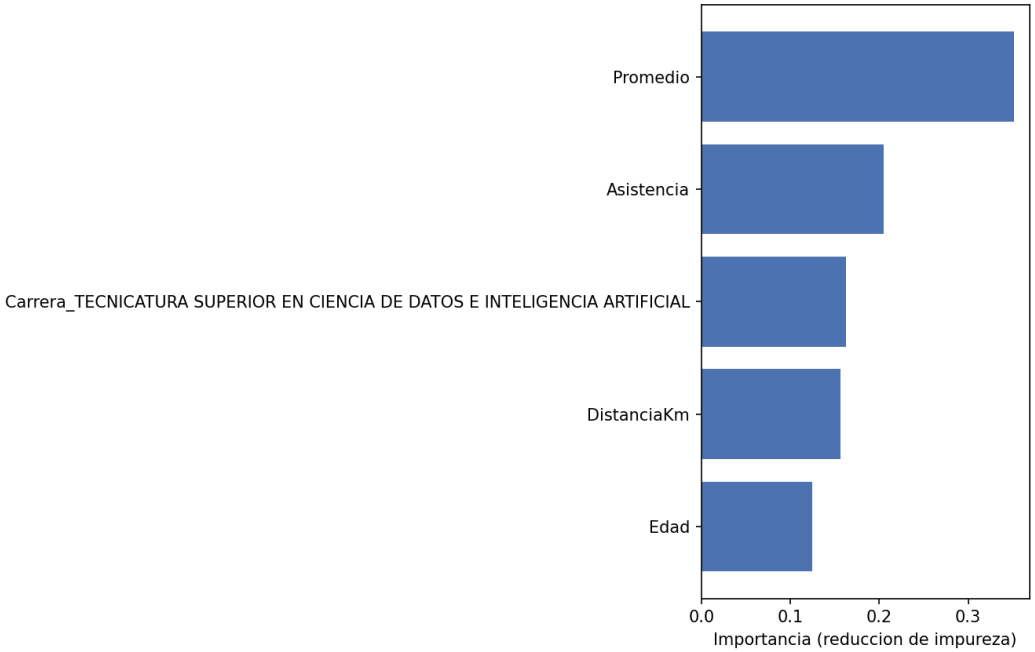


Figura 2. Importancia de las variables según la reducción de impureza del árbol.

Pero esa lectura no se sostiene. La importancia por reducción de impureza está sesgada a favor de las variables continuas con muchos valores distintos —justamente Promedio y Asistencia—. Por eso se calculó además la **importancia por permutación medida sobre el conjunto de prueba**, que es más honesta: desordena una variable y mide cuánto empeora el modelo. El resultado es demoledor: ninguna variable produce una caída apreciable (Carrera +0,007, Asistencia +0,001, el resto exactamente 0) y dos dan valores **negativos** —DistanciaKm −0,009 y Promedio −0,073—, lo que significa que el modelo **mejora si se le destruye la información de esas variables**. Es decir, estaban aportando ruido, no señal.

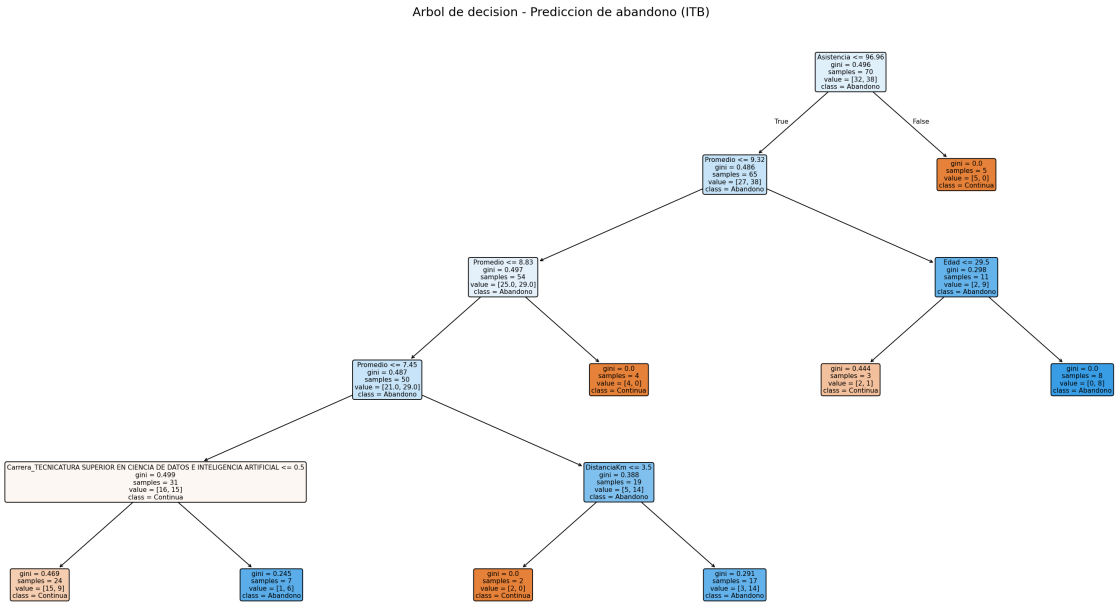


Figura 3. Árbol de decisión final (profundidad 5, 8 hojas). La versión en alta resolución está en salidas/arbol_decision.png.

Conclusión sobre la Tarea 4. Con este dataset **no corresponde afirmar que una variable sea «la más importante»** para predecir el abandono. La raíz y el orden de importancias cambian al cambiar la semilla o la partición, porque no hay un patrón estable que encontrar. Declarar un ganador sería interpretar ruido como si fuera señal. Reconocer esto es, en sí mismo, una respuesta válida de ciencia de datos.

8. Predicción sobre el conjunto DatosFinal

Se aplicó el modelo a los 100 alumnos sin etiqueta. El resultado se guardó en **salidas/Resultados-PrediccionAbandono.xlsx**, con la clase predicha, la probabilidad de abandono y un nivel de riesgo. La distribución fue **59 «Continúa» y 41 «Abandona»**.

Dos detalles de esa salida refuerzan el diagnóstico de los apartados anteriores. Primero, la franja de riesgo **«Medio» quedó vacía**: 59 alumnos en riesgo Bajo, 0 en Medio y 41 en Alto. Esto ocurre porque con *min_samples_leaf* = 1 las hojas del árbol son puras y devuelven probabilidades extremas (0 o 1), no graduadas —un modelo así no permite armar un ranking de prioridades útil—. Segundo, los 10 alumnos con mayor probabilidad de abandono tienen promedios de **9,45 a 9,83**, es decir, son los de mejor rendimiento académico del grupo. Cualquier tutor rechazaría esa lista de inmediato, y con razón.

> Estas predicciones se entregan como **ejercicio técnico** para completar la consigna, **no como un diagnóstico confiable**. Dado el resultado del test de permutación, usarlas para decidir a quién convocar a tutoría equivaldría a sortear los nombres.

9. Experimento de control: ¿y si el problema fuera nuestro código?

Un resultado negativo obliga a descartar que el error esté en la implementación. Para eso se generó un dataset **simulado de 1.000 alumnos** en el que nosotros definimos las relaciones según la bibliografía sobre abandono: menos asistencia, peor promedio, más materias desaprobadas, mayor distancia y trabajar aumentan la probabilidad de abandonar, mientras que asistir a tutorías protege. Se le aplicó exactamente el mismo pipeline.

Métrica	Datos reales del TP	Datos simulados (con señal)
Accuracy	0,433	0,663
Recall (Abandono)	0,312	0,814
F1 (Abandono)	0,370	0,729
ROC-AUC	0,388	0,739
Baseline	0,533	0,557
¿Supera al baseline?	No (–0,100)	Sí (+0,106)

Sobre los datos simulados el mismo código **sí encuentra el patrón**: supera al baseline, alcanza un AUC de 0,739 y el ranking de importancias reproduce los coeficientes que habíamos fijado a mano —Asistencia (0,375), Promedio (0,232), DistanciaKm (0,162), Desaprobadas (0,142), Tutorías y Trabaja—, con **Asistencia en la raíz**. El árbol resultante es claramente interpretable.

> **Qué demuestra el anexo.** El preprocesamiento, el entrenamiento y la evaluación están correctamente implementados: cuando los datos contienen una relación, este mismo pipeline la encuentra. Por contraste, queda establecido que el desempeño pobre con los datos del TP **es una propiedad del dataset y no un error de programación**. Aclaración necesaria: los datos del anexo son inventados y sirven como control metodológico, nunca como conclusión sobre los alumnos del ITB.

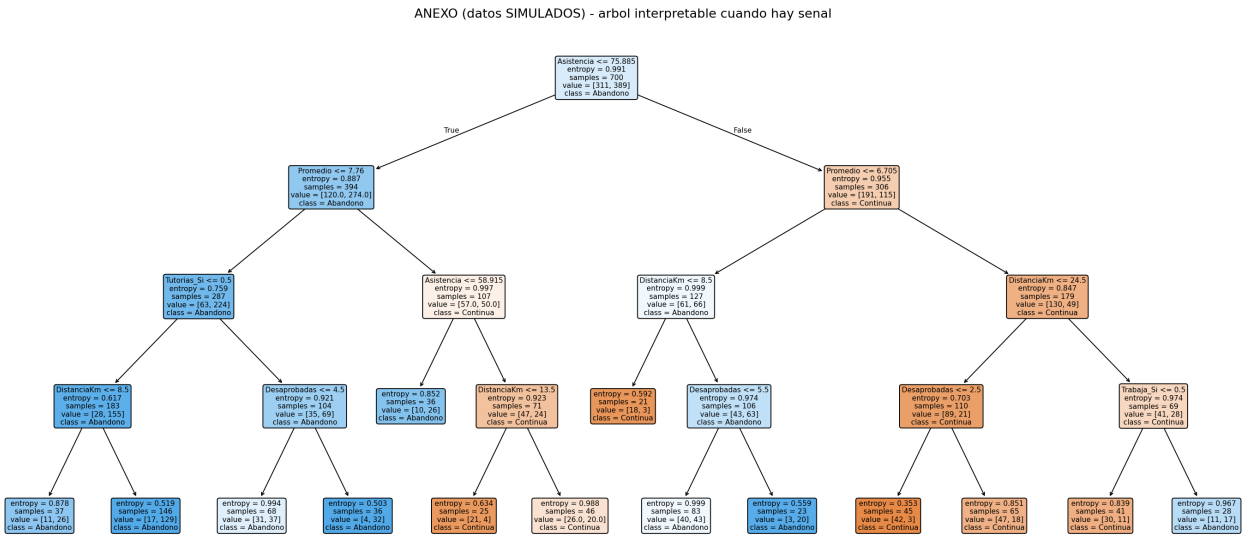


Figura 4. Anexo — árbol entrenado sobre datos simulados. Así se ve un árbol interpretable cuando los datos efectivamente contienen señal.

10. Recomendaciones para el ITB (Tarea 5)

A. Sobre los datos — prioridad 1

- **Auditar el origen del dataset.** El test de permutación indica que los atributos no se relacionan con el estado final. Si los datos fueron generados de forma sintética o aleatoria, ningún algoritmo va a funcionar: el problema no es el modelo, es la materia prima.
- **Ampliar la muestra.** 100 registros son muy pocos para 10 atributos, uno de ellos con 9 categorías que One-Hot expande a más de 20 columnas. Se necesitan varios cientos de casos.
- **Incorporar variables realmente predictoras:** nota de ingreso, materias recursadas, inasistencias *consecutivas* (no solo el promedio anual), abandono de cursada en el primer mes, beca y estado de cuota, horas semanales de trabajo (no solo sí/no), años entre el secundario y el ingreso, turno de cursada y uso del campus virtual.
- **Registrar la etiqueta con más detalle:** no solo «abandonó/continúa» sino *cuándo* y, si se sabe, *por qué*. Eso permite pasar a un modelo de supervivencia, que es lo que sirve para intervenir a tiempo.
- **Definir una fecha de corte.** Hoy los atributos y la etiqueta parecen del mismo período. Un sistema de alerta temprana debe predecir el abandono del 2.º cuatrimestre usando únicamente datos disponibles al cierre del 1.º.

B. Sobre el modelo y su uso

- **No poner este modelo en producción.** Con un rendimiento equivalente al azar, etiquetar alumnos como «en riesgo» sería arbitrario: se los estigmatiza y se malgastan horas de tutoría.
- **Priorizar recall sobre precision** cuando el modelo sí funcione. Es mucho más barato invitar a una tutoría a alguien que no la necesitaba que perder a un alumno que sí estaba en riesgo. En la práctica: bajar el umbral de decisión de 0,50 a ~0,35.
- **Entregar probabilidades y no solo la clase**, para poder ordenar a los alumnos por riesgo y atender primero a los más críticos con los recursos disponibles.
- **No usar Género como variable de decisión** (ni Carrera con criterio punitivo). Son variables sensibles: conviene medirlas para auditar sesgos, nunca para decidir.
- **Comparar contra otros modelos** —regresión logística, Random Forest, Gradient Boosting—. Elegimos el árbol por su interpretabilidad, que es clave para que un tutor entienda y acepte la alerta, pero vale medir cuánta performance se resigna.

C. Institucionales, independientes del modelo

- **Montar un tablero con reglas simples y transparentes** mientras no haya un modelo válido: por ejemplo, «3 inasistencias consecutivas» o «0 materias aprobadas al segundo mes» disparan el llamado de un tutor. Una regla explícita y auditable hoy vale más que un modelo de caja negra sin validar.
- **Cerrar el circuito:** registrar cada intervención —a quién se contactó, qué se hizo y qué resultado tuvo—. Esos registros son el dataset que permitirá, en uno o dos años, entrenar un modelo que sí funcione y además medir el impacto real del programa de acompañamiento.

11. Conclusión

Se cumplieron las cinco tareas de la consigna: los datos se analizaron y prepararon (entropía inicial de 0,9954 bits, dos variables derivadas, One-Hot dentro de un Pipeline y partición estratificada 70/30), se entrenó un árbol de decisión con hiperparámetros elegidos por validación cruzada sobre 360 combinaciones, se lo evaluó en un conjunto de prueba independiente, se intentó interpretarlo y se elaboraron recomendaciones.

El hallazgo central, sin embargo, es negativo: **el árbol no supera al clasificador trivial** (0,433 contra 0,533), su ROC-AUC de 0,388 es peor que el azar y el test de permutación (p = 0,164) confirma que no hay relación

aprendible entre los atributos registrados y el abandono. El experimento de control descarta que sea un problema de implementación.

La lección metodológica que nos llevamos es que **forzar una conclusión habría sido el verdadero error**. Era tentador informar que «la asistencia es la variable más importante» porque aparece en la raíz del árbol, pero la evidencia muestra que esa raíz es producto del azar. En ciencia de datos, reportar honestamente que el dataset no permite responder la pregunta —y explicar por qué, y qué habría que medir para poder responderla— es más valioso que entregar un modelo que parece funcionar y no funciona.

Archivos generados

Archivo (carpeta salidas/)	Contenido
Resultados-PrediccionAbandono.xlsx	Predicciones de los 100 alumnos, métricas e importancias
arbol_decision.png	Árbol de decisión final en alta resolución
matriz_confusion.png	Matriz de confusión del conjunto de prueba
importancia_variables.png	Ranking de importancia de variables
reglas_arbol.txt	Reglas del árbol en formato texto
anexo_arbol_con_senal.png	Árbol del experimento de control con datos simulados